PYTHON · FUNÇÕES



RACIOCÍNIO ALGORÍTMICO

32 exercícios para dominar funções em Python

Fácil · 12

Médio · 12

Difícil · 8

SUMÁRIO

Lista completa de exercícios

32 exercícios organizados em três blocos de dificuldade progressiva.

Nível Fácil

01 Mensagem Personalizada com Moldura

02 Calculador de Área de Terreno

03 Verificador de Par ou Ímpar

04 Contador com Passo Personalizado

05 Fatorial de um Número

06 Analisador Numérico Sem Bibliotecas

07 Validador de Situação Eleitoral

08 Gerador de Tabuada Formatada

09 Descobrindo o Maior Valor com Empacotamento

10 Ficha de Jogador com Parâmetros Opcionais

11 Média Aritmética e Dicionário de Situação

12 Contador de Vogais em Frases

Nível Médio

13 Validação de Dados e Cadastro Continuado

14 Coleta e Estatísticas Numéricas

15 Construtor e Exibidor de Matrizes Tabulares

16 Carrinho de Compras com Cupons de Desconto

17 Boletim de Notas da Turma

18 Sanitização de Texto e Frequência de Palavras

19 Fatorial com Visualização de Passos e Validação

20 Simulador de Caixa Eletrônico

21 Jogo de Adivinhação com Ranking de Tentativas

22 Conversor de Unidades de Temperatura

23 Sanitizador e Validador de Nomes Próprios

24 Desempenho e Ficha de Jogador de Futebol

Nível Difícil

25 Mini-Sistema de Gestão de Alunos

26 Controle de Estoque e Inventário de Loja

27 Análise de Cadastro de Trabalhador e Aposentadoria

28 Gerenciador Financeiro Pessoal

29 Processador Numérico e Ordenação Manual

30 Gerenciador de Tabela de Campeonato

31 Sistema Interativo de Ajuda do Python

32 Sistema de Urna Eletrônica e Apuração de Votos

FÁCIL

Nível Fácil

12 exercícios

01 Mensagem Personalizada com Moldura

Contexto

Em sistemas de terminal, é comum querer destacar títulos e cabeçalhos com molduras visuais ajustáveis para melhorar a experiência do usuário.

`titulo(texto)`

Recebe uma frase qualquer como parâmetro e exibe o texto centralizado entre duas linhas adaptáveis formadas pelo caractere ~. O tamanho da moldura deve acompanhar automaticamente o comprimento do texto digitado, com uma margem de segurança de alguns caracteres em cada ponta.

Exemplo

```
Chamada: titulo('CURSO DE PYTHON')

Saída:
~~~~~~~~~~~~~~~~~~~~
  CURSO DE PYTHON
~~~~~~~~~~~~~~~~~~~~
```

02 Calculador de Área de Terreno

Contexto

Uma imobiliária precisa de uma ferramenta rápida para que seus corretores calculem a área de terrenos retangulares diretamente no terminal.

`area(largura, comprimento)`

Recebe as duas dimensões de um terreno retangular informadas pelo usuário no programa principal, calcula a área total em metros quadrados (m²) e exibe uma mensagem formatada com o resultado.

Requisitos

- O programa principal deve solicitar a largura e o comprimento via input().
- A função deve realizar a multiplicação e printar o resultado formatado com 2 casas decimais.

03 Verificador de Par ou Ímpar

Contexto

Você está desenvolvendo um módulo básico de matemática para alunos do ensino fundamental e precisa de uma função simples que identifique a paridade de qualquer número digitado.

`par_ou_impar(numero)`

Recebe um número inteiro como parâmetro. Não deve printar nada diretamente; em vez disso, deve retornar (return) a string 'PAR' se o número for divisível por 2 ou 'ÍMPAR' caso contrário. No programa principal, o usuário digita um número, a função é chamada e o resultado retornado é impresso na tela.

04 Contador com Passo Personalizado

Contexto

Treinar laços de repetição com contagem progressiva e regressiva é fundamental. Vamos automatizar isso em uma função reutilizável.

`contador_passo(inicio, fim, passo)`

Recebe três valores inteiros e realiza uma contagem na tela mostrando cada número, do valor inicial ao final, respeitando o intervalo (passo) informado.

Requisitos

- Trate o valor de passo: se o usuário passar passo = 0, a função deve assumir passo = 1.
- Se o início for maior que o fim, a contagem deve ser regressiva automaticamente.

05 Fatorial de um Número

Contexto

O cálculo fatorial (n!) é a multiplicação de um número por todos os seus antecessores inteiros positivos. É um exercício clássico para praticar acúmulo de valores dentro de laços.

`fatorial(numero)`

Recebe um número inteiro e calcula o seu fatorial utilizando um laço for ou while. A função deve retornar o valor final calculado.

Exemplo

```
No programa principal, leia um número do teclado e mostre uma frase como:
"O fatorial de 5 é 120."
```

06 Analisador Numérico Sem Bibliotecas

> ***Contexto***
> *Em muitas situações, precisamos extrair propriedades matemáticas básicas de um número sem depender do formato original ou de módulos externos.*

`analisa_numero(n)`

Recebe um número real (float). Deve calcular e exibir diretamente na tela:

Requisitos

- O dobro do número.
- O triplo do número.
- A raiz quadrada, calculada obrigatoriamente usando o operador exponencial ** 0.5, sem importar o módulo math.

07 Validador de Situação Eleitoral

> ***Contexto***
> *O Tribunal Regional Eleitoral precisa verificar rapidamente a obrigatoriedade do voto de cidadãos com base no ano de nascimento.*

`voto(ano_nascimento)`

Recebe o ano de nascimento de uma pessoa. Deve calcular a idade atual (considerando o ano corrente fixado no código, por exemplo, 2026) e retornar uma string indicando a situação:

Requisitos

- 'VOTO NEGADO' para menores de 16 anos.
- 'VOTO OPCIONAL' para pessoas com 16 ou 17 anos ou com 70 anos ou mais.
- 'VOTO OBRIGATÓRIO' para pessoas entre 18 e 69 anos.

08 Gerador de Tabuada Formatada

> ***Contexto***
> *Criar utilitários de estudo é ótimo para praticar formatação de saídas de texto e laços contados.*

`tabuada(num)`

Recebe um número inteiro qualquer. Ao ser executada, a função deve imprimir a tabuada completa desse número (de 1 a 10), utilizando uma formatação bem alinhada e organizada com cabeçalho e rodapé.

09 Descobrindo o Maior Valor com Empacotamento

> ***Contexto***
> *Nem sempre sabemos com antecedência quantos dados o usuário vai enviar para uma função. O Python permite receber um número indeterminado de argumentos através do empacotamento (*args).*

`maior(*numeros)`

Recebe vários parâmetros numéricos inteiros. A função deve analisar todos os valores passados, mostrar quais números foram recebidos, quantos números foram informados ao todo e qual deles é o maior valor.

Exemplo

```
Chamada: maior(2, 9, 4, 5, 7, 1)
→ indica que foram passados 6 valores e o maior é o 9.
```

10 Ficha de Jogador com Parâmetros Opcionais

> ***Contexto***
> *Cadastros muitas vezes lidam com dados incompletos fornecidos pelos usuários, sendo necessário preencher valores padrão (default).*

`ficha(nome='<desconhecido>', gols=0)`

Recebe dois parâmetros opcionais. O programa principal deve ler o nome do jogador e a quantidade de gols marcados. Se o usuário deixar o nome em branco ou não digitar a quantidade de gols, a função deve lidar com esses casos utilizando os valores padrão informados na assinatura da função.

11 Média Aritmética e Dicionário de Situação

> ***Contexto***
> *Escolas precisam gerar boletins onde os dados do aluno ficam agrupados em estruturas organizadas.*

`calcula_media(nota1, nota2)`

Recebe duas notas de um aluno. Deve calcular a média aritmética e retornar um dicionário contendo as seguintes chaves:

Requisitos

- 'Média': valor numérico da média.
- 'Situação': 'Aprovado' caso a média seja maior ou igual a 7.0, ou 'Reprovado' se for menor.

12 Contador de Vogais em Frases

> ***Contexto***
> *Processamento básico de texto envolve percorrer strings caractere por caractere para contagem e verificação.*

`conta_vogais(texto)`

Recebe uma palavra ou frase. Deve converter o texto para minúsculas, percorrer cada caractere e retornar a quantidade total de vogais (a, e, i, o, u) presentes na frase, ignorando acentos e espaços.

MÉDIO

Nível Médio

12 exercícios

13 Validação de Dados e Cadastro Continuado

> ***Contexto***
> *Para criar cadastros confiáveis, precisamos separar a lógica de validação de dados da lógica de armazenamento.*

Desenvolva um programa de cadastro de usuários composto por duas funções:

`valida_idade(msg)`

Exibe a mensagem de solicitação e usa um laço while para garantir que o usuário digite apenas um número inteiro positivo válido, repetindo a leitura até estar correto.

`cadastrar_pessoa()`

Lê o nome do usuário e chama a função valida_idade() para obter a idade, retornando um dicionário contendo {'nome': ..., 'idade': ...}.

Programa principal

Deve chamar cadastrar_pessoa() continuamente e guardar os dicionários em uma lista até que o usuário decida parar (pergunta: "Deseja continuar? [S/N]"). No final, exiba a lista completa de cadastrados.

14 Coleta e Estatísticas Numéricas

> ***Contexto***
> *Agrupar entradas do usuário em listas permite realizar análises estatísticas completas ao término da coleta.*

Crie duas funções para processar uma sequência numérica:

`gerar_lista()`
Lê uma quantidade indeterminada de números digitados pelo usuário até que ele digite um valor de parada (por exemplo, -1) e retorna a lista preenchida.

`analisar_lista(lista)`
Recebe a lista gerada, calcula e imprime na tela o maior número, o menor número, a soma total e a média dos valores.

15 Construtor e Exibidor de Matrizes Tabulares

> ***Contexto***
> *Matrizes em Python são representadas como listas dentro de listas. Manipulá-las exige laços aninhados bem estruturados.*

Construa um programa com duas funções:

`criar_matriz(linhas, colunas)`
Pede ao usuário para digitar o valor de cada posição M[l][c] da matriz e retorna a estrutura de lista composta.

`exibir_matriz(matriz)`
Imprime a matriz na tela em formato tabular (linhas e colunas alinhadas) e exibe o resultado da soma de todos os elementos pertencentes à terceira coluna.

16 Carrinho de Compras com Cupons de Desconto

> ***Contexto***
> *Sistemas de e-commerce possuem etapas bem definidas, como a soma de itens do carrinho e a aplicação de regras promocionais.*

Crie um sistema de checkout com duas funções:

`calcular_total(lista_precos)`
Recebe uma lista de valores numéricos referentes aos produtos comprados e retorna a soma do carrinho.

`aplicar_desconto(total, cupom)`
Recebe o valor total da compra e uma string do cupom. Se o cupom for 'DEZ', aplica 10% de desconto; se for 'VINTE', aplica 20%. Se o cupom for inválido ou vazio, não aplica desconto. Deve retornar o valor final a pagar.

17 Boletim de Notas da Turma

> ***Contexto***
> *Professores precisam coletar notas, analisar o desempenho geral da sala e gerar relatórios visuais claros.*

Desenvolva um programa modularizado com três funções:

`ler_notas()`
Solicita notas de alunos até o usuário encerrar a entrada e retorna uma lista de notas.

`calcular_estatisticas(lista_notas)`
Recebe a lista e retorna um dicionário com o total de notas registradas, a maior nota, a menor nota e a média da turma.

`exibir_boletim(dicionario_estatisticas)`
Recebe o dicionário gerado e imprime um boletim organizado com molduras e títulos.

18 Sanitização de Texto e Frequência de Palavras

> ***Contexto***
> *Ferramentas de análise de texto limpam os dados brutos antes de extrair métricas de frequência.*

Crie três funções para analisar uma frase digitada:

`limpar_texto(texto)`
Recebe a frase e remove pontuações simples (vírgulas e pontos), retornando a string limpa em letras minúsculas.

`separar_palavras(texto_limpo)`
Pega a string e a divide em uma lista de palavras, utilizando o método .split().

`contar_frequencia(lista_palavras)`
Percorre a lista de palavras e retorna um dicionário em que cada chave é uma palavra e o valor é a quantidade de vezes que ela apareceu na frase.

19 Fatorial com Visualização de Passos e Validação

> ***Contexto***
> *Aprimorar o exercício de fatorial permitindo que o usuário escolha ver ou não o cálculo passo a passo na tela.*

Crie duas funções:

`leia_int(msg)`
Função de validação que não aceita entradas vazias ou alfanuméricas, garantindo o retorno de um valor inteiro positivo.

`fatorial(n, show=False)`
Calcula o fatorial de n. Se o parâmetro opcional show for True, deve imprimir a equação na tela (ex: 5 x 4 x 3 x 2 x 1 = 120). Se show for False, deve apenas retornar o resultado numérico.

20 Simulador de Caixa Eletrônico

> ***Contexto***
> *Caixas eletrônicos precisam validar saques e calcular a menor quantidade de cédulas necessárias para entregar o valor solicitado.*

Implemente o fluxo de saque dividido em três funções:

`solicitar_valor_saque()`
Pede o valor que o cliente deseja sacar e garante que seja um valor inteiro positivo.

`calcular_cedulas(valor)`
Realiza a decomposição do valor informando quantas notas de R$ 50, R$ 20, R$ 10 e R$ 1 serão necessárias, retornando um dicionário com essa contagem.

`exibir_saque(dicionario_cedulas)`
Imprime o resumo final informando quantas cédulas de cada valor serão entregues.

21 Jogo de Adivinhação com Ranking de Tentativas

> ***Contexto***
> *Jogos simples em terminal se tornam mais interessantes quando guardam o histórico e ranking dos jogadores.*

Crie um jogo de adivinhação estruturado em três funções, sem usar a biblioteca random. Gere o número secreto por meio de um cálculo matemático com base no tamanho do nome do jogador ou no limite informado:

`gerar_numero_secreto(limite)`
Gera e retorna um número inteiro baseado em operações matemáticas simples no intervalo de 1 a limite.

`jogar_rodada(secreto)`
Gerencia os palpites do jogador via while, dando dicas se o número digitado é maior ou menor que o secreto, e retorna o total de tentativas usadas até acertar.

`salvar_ranking(ranking, nome, tentativas)`
Adiciona o nome do jogador e seu número de tentativas a uma lista de classificação.

22 Conversor de Unidades de Temperatura

> ***Contexto***
> *Aplicativos de utilidades se beneficiam de menus interativos que acionam funções de conversão específicas.*

Escreva três funções para um programa de conversão térmica:

`celsius_para_fahrenheit(c)`
Recebe a temperatura em Celsius e retorna o valor em Fahrenheit (F = C × 1.8 + 32).

`fahrenheit_para_celsius(f)`
Recebe a temperatura em Fahrenheit e retorna o valor em Celsius (C = (F − 32) / 1.8).

`menu_conversor()`
Exibe o menu de opções na tela, lê a escolha do usuário, solicita o valor e chama a função de conversão adequada, mostrando o resultado formatado.

23 Sanitizador e Validador de Nomes Próprios

> ***Contexto***
> *Formulários web frequentemente precisam higienizar nomes digitados incorretamente antes de salvar no banco de dados.*

Desenvolva duas funções de tratamento de strings:

`normalizar_nome(nome)`
Remove espaços extras no início/fim e entre as palavras, retornando a string com cada parte do nome capitalizada (Title Case).

`validar_nome_completo(nome_normalizado)`
Verifica se o texto contém pelo menos duas palavras (nome e sobrenome). Retorna True se for um nome completo válido ou False se o usuário tiver digitado apenas o primeiro nome.

24 Desempenho e Ficha de Jogador de Futebol

> ***Contexto***
> *Agrupamento de dados esportivos utilizando dicionários e funções para análise de aproveitamento individual.*

Crie três funções para registrar dados de atletas:

`cadastrar_jogador()`
Lê o nome do jogador e a quantidade de partidas disputadas, cria uma lista com a quantidade de gols marcados em cada partida e retorna um dicionário completo.

`calcular_media_gols(jogador_dict)`
Recebe o dicionário do jogador e calcula a média de gols por partida.

`exibir_ficha_jogador(jogador_dict)`
Mostra um resumo formatado contendo o total de gols, as partidas disputadas e a média calculada.

DIFÍCIL

Nível Difícil

8 exercícios

25 Mini-Sistema de Gestão de Alunos

> ***Contexto***
> *Um sistema acadêmico completo para pequenas escolas, que exige modularização completa para o gerenciamento das turmas.*

Desenvolva um programa de gestão de alunos dividido obrigatoriamente nas seguintes 5 funções:

`exibir_cabecalho(texto)`
Imprime cabeçalhos visuais padronizados com molduras adaptáveis.

`cadastrar_aluno()`
Lê o nome do aluno e 3 notas, retornando um dicionário contendo {'nome': ..., 'notas': [...]}.

`calcular_media(lista_notas)`
Recebe a lista com as 3 notas e retorna o valor da média aritmética.

`definir_situacao(media)`
Retorna a string 'APROVADO' (média ≥ 7.0), 'RECUPERAÇÃO' (média entre 5.0 e 6.9) ou 'REPROVADO' (média < 5.0).

`exibir_relatorio_turma(lista_alunos)`
Percorre a lista de todos os alunos cadastrados e imprime uma tabela formatada com nome, notas, média e situação de cada um, além da média geral da turma no rodapé.

26 Controle de Estoque e Inventário de Loja

> ***Contexto***
> *Gerenciadores de estoque exigem funções de cadastro, atualização e relatórios financeiros consolidados.*

Crie um sistema de inventário de produtos estruturado nas seguintes 5 funções:

`adicionar_produto(estoque)`
Lê código, nome, preço unitário e quantidade de um produto, salvando esses dados em um dicionário dentro da lista de estoque.

`atualizar_quantidade(estoque)`
Solicita o código de um produto e a nova quantidade, atualizando o registro existente.

`calcular_valor_total_estoque(estoque)`
Calcula o valor financeiro acumulado de todos os produtos no estoque (soma de preço × quantidade).

`buscar_mais_caro(estoque)`
Identifica e retorna o dicionário do produto de maior preço unitário.

`exibir_estoque_completo(estoque)`
Imprime uma tabela alinhada com todos os itens do inventário, suas quantidades, preços e o patrimônio total da loja.

27 Análise de Cadastro de Trabalhador e Aposentadoria

> ***Contexto***
> *Baseado no clássico exercício do Guanabara, este programa calcula dados trabalhistas estendidos e a projeção de aposentadoria.*

Construa um sistema de análise previdenciária dividido em 5 funções:

`ler_dados_trabalhador()`
Coleta nome, ano de nascimento e carteira de trabalho (CTPS). Se a CTPS for diferente de 0, solicita também o ano de contratação e o salário.

`calcular_idade(ano_nascimento, ano_atual)`
Calcula e retorna a idade do trabalhador.

`calcular_tempo_aposentadoria(idade, ano_contratacao, ano_atual)`
Calcula com quantos anos a pessoa se aposentará, considerando a regra de 35 anos de contribuição a partir do ano de contratação.

`formatar_moeda(valor)`
Recebe um número float e o retorna formatado como string no padrão de moeda (ex: R$ 2.500,00).

`exibir_ficha_trabalhador(trabalhador_dict)`
Consolida todas as informações obtidas e exibe uma ficha detalhada com todas as chaves e valores.

28 Gerenciador Financeiro Pessoal

> ***Contexto***
> *Aplicativos de finanças pessoais dependem de rotinas para validação de valores positivos, separação de entradas/saídas e cálculo de saldo.*

Escreva um programa de controle de orçamento composto por 5 funções:

`validar_valor_positivo(msg)`
Garante que o usuário digite um número real estritamente maior que zero.

`registrar_receita()`
Solicita o nome da fonte de renda e o valor (usando a validação) e retorna o dicionário da receita.

`registrar_despesa()`
Solicita a categoria do gasto e o valor gasto, retornando o dicionário da despesa.

`calcular_total_categoria(lista_despesas)`
Calcula e exibe a soma dos gastos agrupados por categoria.

`exibir_balanco_geral(lista_receitas, lista_despesas)`
Calcula o total de receitas, o total de despesas e o saldo final (receitas − despesas), exibindo um relatório que alerta se a conta está no 'VERDE' ou no 'VERMELHO'.

29 Processador Numérico e Ordenação Manual

> ***Contexto***
> *Entender a lógica de ordenação sem depender de métodos nativos prontos (como .sort()) consolida a compreensão de laços e trocas de variáveis.*

Crie um utilitário de tratamento de listas composto por 5 funções:

`ler_lista_inteiros()`
Coleta números digitados pelo usuário até que ele digite 0 para encerrar, retornando a lista original.

`filtrar_pares(lista)`
Gera e retorna uma nova lista contendo apenas os números pares da lista original.

`filtrar_impares(lista)`
Gera e retorna uma nova lista contendo apenas os números ímpares.

`ordenar_lista_manual(lista)`
Ordena os elementos de uma lista numérica em ordem crescente sem usar .sort() ou sorted(), utilizando laços for ou while com trocas manuais de posição.

`exibir_resumo_processamento(original, pares, impares, ordenada)`
Exibe todas as listas processadas na tela com formatação clara.

30 Gerenciador de Tabela de Campeonato

> ***Contexto***
> *Tabelas esportivas atualizam pontuações, saldo de gols e classificações dinamicamente após a digitação dos resultados.*

Construa um sistema de classificação de times de futebol com 5 funções:

`cadastrar_times()`
Lê os nomes de 4 times e inicializa a estrutura de dados para cada um (pontos, vitórias, empates, derrotas).

`registrar_partida(times_dict)`
Lê os gols marcados em um confronto entre dois times e atualiza as estatísticas de cada um.

`calcular_pontuacao(vitorias, empates)`
Retorna o total de pontos acumulados (3 × vitórias + 1 × empates).

`ordenar_classificacao(times_list)`
Ordena a lista de times do maior para o menor total de pontos acumulados.

`exibir_tabela_campeonato(times_ordenados)`
Imprime a tabela do campeonato organizada em colunas: posição, time, pontos, vitórias, empates e derrotas.

31 Sistema Interativo de Ajuda do Python

> ***Contexto***
> *Inspirado no famoso Exercício 106 do Curso em Vídeo, este programa cria um mini-gerenciador de documentação com menu contínuo.*

Desenvolva um assistente de ajuda interativa para comandos do Python utilizando 5 funções:

`exibir_titulo(msg, simbolo='~')`
Cria molduras personalizadas dinâmicas para destacar mensagens do sistema.

`validar_comando(comando)`
Verifica se a string digitada pelo usuário não está vazia.

`obter_ajuda(funcao_ou_biblioteca)`
Executa a função nativa help() do Python para a função ou comando informado.

`exibir_menu_opcoes()`
Exibe o menu principal na tela e lê a escolha do usuário.

`executar_sistema_ajuda()`
Função principal contendo o laço while que mantém o programa em execução até que o usuário digite a palavra 'FIM'.

32 Sistema de Urna Eletrônica e Apuração de Votos

> ***Contexto***
> *A simulação de uma votação exige registro seguro de votos, tratamento de opções nulas/brancas e apuração percentual.*

Desenvolva um programa de simulação de urna eleitoral contendo 5 funções:

`inicializar_urna()`
Cria o dicionário de candidatos (ex: Candidato 1, Candidato 2, Nulos, Brancos) com contadores zerados.

`registrar_voto(urna, opcao)`
Recebe a opção escolhida pelo eleitor e incrementa o contador correspondente na estrutura de dados da urna.

`calcular_percentual(votos_opcao, total_votos)`
Calcula e retorna a porcentagem de votos que determinada opção obteve em relação ao total.

`determinar_vencedor(urna)`
Analisa a contagem dos candidatos válidos e retorna o nome do candidato mais votado.

`exibir_boletim_urna(urna)`
Imprime o boletim de apuração detalhado contendo a quantidade absoluta de votos de cada opção, o percentual de cada uma e o resultado final da eleição, anunciando o vencedor.